

VAEs for Video Generation Models

1. Compression Parameters and DiT Token Counts

Compression factors are listed as **time × height × width**, and C denotes the number of VAE channels. The final column gives the shape **after DiT patchification and before projection**, with the elements within each patch folded into the channel dimension.

$$R_{\text{nom}} = \frac{3r_t r_h r_w}{C}, \quad R_{\text{actual}} = \frac{3THW}{C T_z H_z W_z}, \quad N = \frac{T_z H_z W_z}{p_t p_h p_w}.$$

The DiT patch size is $p_t \times p_h \times p_w$; patchification only rearranges elements. Compression ratios are calculated from the VAE output and already account for patchification inside the VAE. Input dimensions must be aligned.

Model	VAE	Compression time × height × width	Latent channels C	Element compression ratio R_{nom}	DiT patchify	DiT input shape
MiniMax H3	H3-VisualVAE Causal 3D CNN encoder + non-causal ViT decoder ; reconstructs spatiotemporal pixel blocks in a single projection.	$4 \times 16 \times 16$	24	128:1	$1 \times 2 \times 2$	$[B, 96, T_{\text{H3}}, \frac{H}{32}, \frac{W}{32}]$
LTX-2.5	LTX-2.5 Video VAE Causal 3D CNN encoder + non-causal 3D CNN / NA Transformer diffusion decoder (CNN in its default configuration); spatial patchify ×4; the diffusion variant uses one-step pixel denoising.	$8 \times 32 \times 32$	128	192:1	$1 \times 1 \times 1$	$[B, 128, 1 + \frac{T-1}{8}, \frac{H}{32}, \frac{W}{32}]$
FLUX 3 Action	FLUX 3 Video VAE Causal NA Transformer encoder + non-causal NA Transformer decoder ; 5×5×5 neighborhood attention.	$4 \times 32 \times 32$	96	128:1	$1 \times 1 \times 1$	$[B, 96, 1 + \frac{T-1}{4}, \frac{H}{32}, \frac{W}{32}]$
Wan 2.1 / 2.2 A14B / Lingbot-Video	Wan2.1-VAE Causal 3D CNN encoder + causal 3D CNN decoder; within-frame spatial attention and per-layer history caches.	$4 \times 8 \times 8$	16	48:1	$1 \times 2 \times 2$	$[B, 64, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$
Wan 2.2 TI2V-5B	Wan2.2-VAE Causal 3D CNN encoder + causal 3D CNN decoder; spatial patchify ×2 and residual shortcuts in resampling modules.	$4 \times 16 \times 16$	48	64:1	$1 \times 2 \times 2$	$[B, 192, 1 + \frac{T-1}{4}, \frac{H}{32}, \frac{W}{32}]$
HunyuanVideo-1.5	AutoencoderKLConv3D Causal 3D CNN encoder + causal 3D CNN decoder; causal attention at the bottleneck and residual 3D pixel shuffle.	$4 \times 16 \times 16$	32	96:1	$1 \times 1 \times 1$	$[B, 32, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$
MAGI-2 Preview	Wan2.2-VAE + TurboVAED Causal 3D CNN encoder + non-causal distilled 3D CNN decoder .	$4 \times 16 \times 16$	48	64:1	$1 \times 1 \times 1$	$[B, 48, 1 + \frac{T-1}{4}, \frac{H}{16}, \frac{W}{16}]$

H3 handles temporal compression somewhat differently: strictly speaking, its ratio is not 4:1, and its element compression ratio is also somewhat lower than 128:1.

2. Wan2.1-VAE

We now take a closer look at what is probably the most classic VAE: Wan2.1-VAE. Consider an RGB video with **81 frames at 480×832**:

Full video: [3, 81, 480, 832] → **latent:** [16, 21, 60, 104] → **reconstructed video:** [3, 81, 480, 832].

$$T_z = 1 + \frac{81 - 1}{4} = 21, \quad H_z = \frac{480}{8} = 60, \quad W_z = \frac{832}{8} = 104.$$

2.1 Encoder: Tracing a Subsequent 4-Frame Video Chunk

The full video is divided into **21 chunks of 1, 4, 4, ... frames**: one initial single-frame chunk followed by 20 subsequent chunks. Wan2.1-VAE uses a Feature Cache to retain historical features from earlier chunks. We use one subsequent chunk to trace its forward pass:

Step	Module and operation	Output shape
Input	Current video chunk	[3, 4, 480, 832]
Entry	Causal convolution, 3→96	[96, 4, 480, 832]
Stage 1	Residual block ×2, outputting 96 channels → spatial downsampling	[96, 4, 240, 416]
Stage 2	Residual block ×2, outputting 192 channels → spatiotemporal downsampling	[192, 2, 120, 208]
Stage 3	Residual block ×2, outputting 384 channels → spatiotemporal downsampling	[384, 1, 60, 104]
Stage 4	Residual block ×2, outputting 384 channels	[384, 1, 60, 104]
Bottleneck	Residual block → spatial Attention → residual block	[384, 1, 60, 104]
Exit	RMSNorm → SiLU → causal convolution, 384→32	[32, 1, 60, 104]

After all 21 chunks have been encoded, their outputs are concatenated along the time dimension, passed through a 1×1×1 convolution, and split into μ and $\log_var$, each with shape [16, 21, 60, 104]. The mean μ is standardized per channel for use as input.

2.2 Decoder: Tracing a Subsequent Latent Time Step

First, **undo the standardization** of the full latent, then apply a **1×1×1 convolution, 16→16**; the shape remains [16, 21, 60, 104]. Latent temporal positions are then decoded one at a time. The table traces a subsequent time step:

Step	Module and operation	Output shape
Input	Latent chunk after undoing standardization	[16, 1, 60, 104]
Entry	Causal convolution, 16→384	[384, 1, 60, 104]
Bottleneck	Residual block → spatial Attention → residual block	[384, 1, 60, 104]
Stage 1	Residual block ×3, outputting 384 channels → spatiotemporal upsampling, 384→192	[192, 2, 120, 208]
Stage 2	Residual block ×3, first increasing 192 to 384 → spatiotemporal upsampling, 384→192	[192, 4, 240, 416]
Stage 3	Residual block ×3, outputting 192 channels → spatial upsampling, 192→96	[96, 4, 480, 832]
Stage 4	Residual block ×3, outputting 96 channels	[96, 4, 480, 832]
Exit	RMSNorm → SiLU → causal convolution, 96→3	[3, 4, 480, 832]

The first frame skips temporal resampling and produces only **1 frame**.

2.3 Downsampling and Upsampling

Operation	Implementation	Shape change
Spatial downsampling	Pad each frame by 1 on the right and bottom $\rightarrow 3 \times 3$ Conv2d, stride=2	Halve H, W ; C, T unchanged
Temporal downsampling	Concatenate 1 cached historical frame $\rightarrow 3 \times 1 \times 1$ temporal convolution with temporal stride 2	Halve T for subsequent chunks; C, H, W unchanged
Spatial upsampling	Per-frame nearest-neighbor interpolation $\times 2$ (nearest-exact) $\rightarrow 3 \times 3$ Conv2d, $C \rightarrow C/2$	Double H, W and halve C ; T unchanged
Temporal upsampling	$3 \times 1 \times 1$ causal convolution, $C \rightarrow 2C \rightarrow$ rearrange the two channel groups into the time dimension	Double T for subsequent chunks; final C, H, W unchanged

Execution order: spatiotemporal downsampling is **spatial first, then temporal**; spatiotemporal upsampling is **temporal first, then spatial**.

Residual Blocks, Caches, and Attention

- **Residual blocks:** the main branch applies two sets of “RMSNorm $\rightarrow$ SiLU $\rightarrow 3 \times 3 \times 3$ causal convolution,” then adds the shortcut. When channel counts differ, the shortcut uses a $1 \times 1 \times 1$ convolution to align them. Residual blocks preserve T, H, W .
- **Causal convolution caches:** a standard causal convolution with temporal kernel size 3 and stride 1 needs features from the preceding **2 temporal positions**. Missing positions at the beginning of the sequence are zero-padded. Each layer maintains its own cache, with temporal positions measured at that layer’s resolution. Temporal downsampling in the table above uses a separate **1-frame cache** path.
- **Within-frame spatial Attention:** attention operates only on the 2D feature map at the same temporal position and preserves $[C, T, H, W]$. Information across time is conveyed through causal convolutions.

With Wan2.1-VAE as a reference, we briefly introduce some follow-up models.

Wan2.2-VAE retains Wan2.1-VAE’s causal convolutions and $4n + 1$ handling, with a temporal compression factor of 4. The main change is the addition of 2×2 patchification at the entry, rearranging spatial positions into the channel dimension. Three subsequent spatial downsampling stages increase spatial compression from 8×8 to 16×16 , while the latent channel count rises from 16 to 48. Architecturally, Wan2.2 widens both the encoder and the decoder: the encoder’s base channel count increases from 96 to 160 and its bottleneck from 384 to 640; the decoder’s base channel count increases from 96 to 256 and its bottleneck from 384 to 1024. In addition to the existing residual blocks, shortcuts are added around entire resampling stages: downsampling aligns shapes through rearrangement and Avgpool, while upsampling uses rearrangement and copying, before adding the result to the main branch.

HunyuanVideo-1.5 keeps $4 \times$ temporal compression, increases compression along both spatial axes from $8 \times$ to $16 \times$, and raises the latent channel count from 16 to 32. Compared with Wan’s separate spatial and temporal resampling, it uses causal convolution with PixelShuffle/Unshuffle-style rearrangement and adds residual shortcuts to the resampling modules and the projections on both sides of the latent. Attention in the middle of the network expands from within-frame spatial attention to spatiotemporal causal attention, allowing direct attention to current and historical frames for cross-frame information exchange.

3. MiniMax-H3 VAE

Next is MiniMax-H3 VAE. MiniMax H3's default video chunking rule requires the length to align with $17n + 5$ (H3's handling here is rather curious; I have considered some possible reasons, but have not yet found an official explanation). If the original video has **81 frames** ($4d + 1$), this example first **repeats the final frame to pad the video to 90 frames**, then removes the final 9 frames after reconstruction.

Full video: $[3, 90, 480, 832] \rightarrow$ **latent:** $[24, 27, 30, 52] \rightarrow$ **reconstructed video:** $[3, 90, 480, 832]$.

$$90 = 17 \times 5 + 5, \quad T_z = 5 \times 5 + 2 = 27, \quad H_z = \frac{480}{16} = 30, \quad W_z = \frac{832}{16} = 52.$$

3.1 Encoder: Tracing a 17-Frame Video Chunk

During encoding, the 90-frame video is internally padded again by repeating the final frame, temporarily reaching **102 frames**. It is then split into **six 17-frame chunks**, each encoded independently.

Step	Module and operation	Output shape
Input	Current video chunk	$[3, 17, 480, 832]$
Entry	$3 \times 3 \times 3$ causal convolution, $3 \rightarrow 128$	$[128, 17, 480, 832]$
Stage 1	Residual block $\times 2$, outputting 128 channels $\rightarrow$ spatial downsampling	$[128, 17, 240, 416]$
Stage 2	Residual block $\times 2$, outputting 256 channels $\rightarrow$ spatiotemporal downsampling	$[256, 9, 120, 208]$
Stage 3	Residual block $\times 2$, outputting 256 channels $\rightarrow$ spatiotemporal downsampling	$[256, 5, 60, 104]$
Stage 4	Residual block $\times 2$, outputting 512 channels $\rightarrow$ spatial downsampling	$[512, 5, 30, 52]$
Stage 5	Residual block $\times 2$, outputting 512 channels	$[512, 5, 30, 52]$
Stage 6	Residual block $\times 2$, outputting 1024 channels	$[1024, 5, 30, 52]$
Exit	GroupNorm $\rightarrow$ SiLU $\rightarrow$ causal convolution, $1024 \rightarrow 48$	$[48, 5, 30, 52]$
Distribution projection	$1 \times 1 \times 1$ convolution, $48 \rightarrow 48$	$[48, 5, 30, 52]$

Both temporal downsampling steps round up, giving the sequence $17 \rightarrow 9 \rightarrow 5$. After all 6 chunks have been encoded, concatenating along time gives $[48, 30, 30, 52]$. With `token_drop=3`, **3 temporal positions are dropped from the end of the full sequence**, leaving $[48, 27, 30, 52]$ (the discarded portion corresponds to the 12 repeated final frames).

The output is then split into μ and $\log_var$, each with shape $[24, 27, 30, 52]$. The latent is sampled from the posterior distribution and then standardized per channel.

3.2 Decoder: Tracing a Latent Window

First, **undo the standardization** of the full latent. Decode it in **windows of 7 temporal positions, advancing by 5 positions each time**, so neighboring windows overlap by 2 positions. We trace one window of shape $[24, 7, 30, 52]$; its video token count and hidden dimension are:

$$N = 7 \times 30 \times 52 = 10920, \quad D = 32 \times 64 = 2048.$$

Step	Module and operation	Output shape
Input	Latent window after undoing standardization	$[24, 7, 30, 52]$
Entry	$1 \times 1 \times 1$ convolution, $24 \rightarrow 24$	$[24, 7, 30, 52]$
Flatten	Turn each spatiotemporal position into one token	$[10920, 24]$
Feature projection	Linear, $24 \rightarrow 2048$	$[10920, 2048]$
Extra tokens	Append 4 register tokens + 1 initially zero token (a classic CV move)	$[10925, 2048]$
Backbone	Transformer block $\times 36$, 3D RoPE	$[10925, 2048]$

Step	Module and operation	Output shape
Exit	LayerNorm $\rightarrow$ Linear, 2048$\rightarrow$3072	[10925, 3072]
Remove extra tokens	Keep the video tokens	[10920, 3072]
Pixel rearrangement	Expand each token into a 4$\times$16$\times$16 RGB block	[3, 28, 480, 832]

The output projection maps each video token to $3072 = 3 \times 4 \times 16 \times 16$ values, then rearranges them into a 4 $\times$ 16 $\times$ 16 RGB block. A window therefore first produces [3, 28, 480, 832], followed by temporal cropping and window blending.

Temporal Cropping and Full-Sequence Stitching

Each window's 28 output frames are split into the first **20 frames** and the final **8 frames**. The first 3 frames of each part are removed, leaving a **17-frame main segment + a 5-frame overlap region**.

The 27 latent temporal positions form **5 windows**. With zero-based indexing, their ranges are 0-6, 5-11, 10-16, 15-21, and 20-26. Each window retains 22 frames, and adjacent windows blend their 5 overlapping frames, yielding:

$$5 \times 22 - 4 \times 5 = 90 \text{ frames.}$$

The blended output has shape [3, 90, 480, 832]. Removing the final 9 frames added in this example restores [3, 81, 480, 832].

3.3 Downsampling and Pixel Expansion

Operation	Implementation	Shape change
Spatial downsampling	Reflection-pad by 1 on the right and bottom $\rightarrow$ 3 $\times$ 3 $\times$ 3 causal convolution, stride (1, 2, 2)	Halve H, W ; C, T unchanged
Spatiotemporal downsampling	Reflection-pad by 1 on the right and bottom $\rightarrow$ 3 $\times$ 3 $\times$ 3 causal convolution, stride (2, 2, 2)	Halve $H, W; T \rightarrow \lceil T/2 \rceil$; C unchanged
Pixel expansion	Linear $\rightarrow$ spatiotemporal rearrangement	Each video token produces one 4 $\times$ 16 $\times$ 16 RGB block

Downsampling order: spatial $\rightarrow$ spatiotemporal $\rightarrow$ spatiotemporal $\rightarrow$ spatial, at the ends of the first four stages, respectively. Temporal convolutions in both downsampling types use 2 zero-padded positions on the left.

Residual Blocks, Normalization, and Attention

- **Residual blocks:** the main branch applies two sets of "GroupNorm $\rightarrow$ SiLU $\rightarrow$ 3 $\times$ 3 $\times$ 3 causal convolution," then adds the shortcut. When channel counts differ, the shortcut uses a 1 $\times$ 1 $\times$ 1 convolution to align them. Residual blocks preserve T, H, W .
- **Causal encoding:** temporal convolutions use zero-padding only on the left, and GroupNorm is computed independently at each temporal position. Each 17-frame chunk enters the Encoder independently.
- **ViT decoder blocks:** each block applies RMSNorm $\rightarrow$ Attention, followed by RMSNorm $\rightarrow$ gated SiLU FFN. Both branches are scaled by learnable parameters before being added back to the residual, preserving $[N, 2048]$.
- **Attention scope:** the ViT Decoder in the released configuration is non-causal and can attend to all temporal positions within the current window.

4. LTX-2.5 Video VAE

Next is LTX-2.5's video VAE. It uses a **causal 3D CNN Encoder** and offers two Decoders: a **non-causal 3D CNN decoder** and a **non-causal NA Transformer diffusion decoder**. Both share the same latent space, with a temporal compression factor of **8**, spatial compression factors of **32** along both height and width, and **128** latent channels.

Again, consider an RGB video with **81 frames at 480×832**. Shapes are written as $[C, T, H, W]$, omitting $B=1$:

Full video: $[3, 81, 480, 832] \rightarrow$ **latent:** $[128, 11, 15, 26] \rightarrow$ **reconstructed video:** $[3, 81, 480, 832]$.

$$T_z = 1 + \frac{81 - 1}{8} = 11, \quad H_z = \frac{480}{32} = 15, \quad W_z = \frac{832}{32} = 26.$$

We trace the forward pass of the full video below. Chunking and overlap blending are separate execution strategies; we do not treat the video as 11 independently encoded and decoded chunks from the outset.

4.1 Encoder: Tracing the Full 81-Frame Video

The entry first applies **4×4 spatial patchification**, rearranging the 16 neighboring pixel positions in each frame into the channel dimension while keeping the temporal length unchanged. Four downsampling stages follow: **spatial** $\rightarrow$ **temporal** $\rightarrow$ **spatiotemporal** $\rightarrow$ **spatiotemporal**.

Step	Module and operation	Output shape
Input	RGB video	$[3, 81, 480, 832]$
Patchify	Spatial 4×4 rearrangement, 3 $\rightarrow$ 48	$[48, 81, 120, 208]$
Entry	3×3×3 causal convolution, 48 $\rightarrow$ 128	$[128, 81, 120, 208]$
Stage 1	Residual block ×4, 128 channels $\rightarrow$ spatial downsampling , 128 $\rightarrow$ 256	$[256, 81, 60, 104]$
Stage 2	Residual block ×6, 256 channels $\rightarrow$ temporal downsampling , 256 $\rightarrow$ 512	$[512, 41, 60, 104]$
Stage 3	Residual block ×4, 512 channels $\rightarrow$ spatiotemporal downsampling , 512 $\rightarrow$ 1024	$[1024, 21, 30, 52]$
Stage 4	Residual block ×2, 1024 channels $\rightarrow$ spatiotemporal downsampling , 1024 $\rightarrow$ 1024	$[1024, 11, 15, 26]$
Bottleneck	Residual block ×2, 1024 channels	$[1024, 11, 15, 26]$
Exit	PixelNorm $\rightarrow$ SiLU $\rightarrow$ causal convolution, 1024 $\rightarrow$ 129	$[129, 11, 15, 26]$

All three temporal downsampling steps preserve the position corresponding to the first frame. The temporal lengths are:

$$81 \rightarrow 41 \rightarrow 21 \rightarrow 11.$$

The first **128 channels of the convolutional head's output are μ** . As in the other examples, normalization using per-channel means and standard deviations gives $[128, 11, 15, 26]$. Unlike the other VAEs, this one has no additional $1\times 1\times 1$ distribution projection convolution.

4.2 CNN Decoder: Reconstructing the Video Stage by Stage

First undo the latent's per-channel normalization, then apply convolutional residual blocks and four upsampling stages. Finally, unpatchify restores the RGB pixels.

Step	Module and operation	Output shape
Input	Latent after undoing normalization	[128, 11, 15, 26]
Entry	$3 \times 3 \times 3$ convolution, 128 → 1024	[1024, 11, 15, 26]
Stage 1	Residual block $\times 2$, 1024 channels → spatiotemporal upsampling , 1024 → 512	[512, 21, 30, 52]
Stage 2	Residual block $\times 2$, 512 channels → spatiotemporal upsampling , 512 → 512	[512, 41, 60, 104]
Stage 3	Residual block $\times 4$, 512 channels → temporal upsampling , 512 → 256	[256, 81, 60, 104]
Stage 4	Residual block $\times 6$, 256 channels → spatial upsampling , 256 → 128	[128, 81, 120, 208]
Stage 5	Residual block $\times 4$, 128 channels	[128, 81, 120, 208]
Exit	PixelNorm → SiLU → $3 \times 3 \times 3$ convolution, 128 → 48	[48, 81, 120, 208]
Unpatchify	Rearrange channels into 4×4 spatial pixel blocks, 48 → 3	[3, 81, 480, 832]

Temporal upsampling first doubles the length and then removes the first temporal position, so each step follows $T \rightarrow 2T - 1$:

$$11 \rightarrow 21 \rightarrow 41 \rightarrow 81.$$

The convolutions in this Decoder run in **non-causal mode**, allowing them to use information from both preceding and subsequent temporal positions.

4.3 Diffusion Decoder: Conditioning Features and Pixel Denoising

This decoder first upsamples the latent into conditioning features, or **context**. Conditioned on this context, it reconstructs the video from pixel noise in **1 denoising step**, directly predicting the reconstructed video x_0 .

Generating Conditioning Features

The official implementation repeats the final latent temporal position **2 additional times** to handle the end boundary of neighborhood Attention. We include this temporary padding in the shapes below and crop it away at the end.

Step	Module and operation	Output shape
Input	Latent after undoing normalization	[128, 11, 15, 26]
Boundary handling	Repeat the final latent temporal position 2 additional times	[128, 13, 15, 26]
Entry	Position-wise Linear, 128 → 2048	[2048, 13, 15, 26]
Stage 1	NA Block $\times 4$ → spatial upsampling , 2048 → 1024	[1024, 13, 30, 52]
Stage 2	NA Block $\times 6$ → temporal upsampling , 1024 → 512	[512, 25, 30, 52]
Stage 3	NA Block $\times 4$ → spatiotemporal upsampling , 512 → 512	[512, 49, 60, 104]
Stage 4	NA Block $\times 2$ → spatiotemporal upsampling , 512 → 256	[256, 97, 120, 208]
Crop	Remove the final 16 temporal positions corresponding to the temporary padding	[256, 81, 120, 208]

The 2 added latent temporal positions undergo $8 \times$ temporal expansion, corresponding to **16 temporal positions**, so cropping gives $97 - 16 = 81$. The resulting context already has the target video's temporal length, while its spatial resolution remains $1/4 \times 1/4$ of the target.

Pixel Denoising and Reconstruction

Initialize random pixel noise with the same shape as the target video. We trace the pixel feature branch below; context is injected as conditioning in each diffusion block.

Step	Module and operation	Output shape
Input	Random pixel noise x_t	[3, 81, 480, 832]
Patchify	Spatial 4×4 rearrangement, 3→48	[48, 81, 120, 208]
Entry	Position-wise Linear, 48→256	[256, 81, 120, 208]
Denoising backbone	Diffusion NA Block ×8, injecting context and denoising timestep conditioning	[256, 81, 120, 208]
Exit	RMSNorm → Linear, 256→48	[48, 81, 120, 208]
Unpatchify	Rearrange channels into 4×4 RGB pixel blocks, 48→3	[3, 81, 480, 832]

The final output is the reconstructed video x_0 . The **8 blocks** refer to **network depth**; the “denoising timestep” indicates the noise level and is distinct from the video’s 81 temporal positions.

4.4 NA Attention: Local Spatiotemporal Attention

NA stands for Neighborhood Attention. At each feature-map position (t, h, w) , it takes only the K and V values within a nearby 3D window and computes Attention with the Q at the current position. The window slides with the query position, and neighboring windows overlap.

For example, away from boundaries, a **3×7×7** window covers the **preceding, current, and following temporal positions**, taking **7×7** spatial grid points at each one. Each query in each attention head therefore attends to $3 \times 7 \times 7 = 147$ positions, including itself.

Stage	Feature channels	Attention heads	Head dimension	Window: time × height × width	Positions per head
Stage 1	2048	32	64	3×7×7	147
Stage 2	1024	16	64	3×7×7	147
Stage 3	512	8	64	3×5×5	75
Stage 4	512	8	64	3×5×5	75
Stage 5: pixel denoising	256	4	64	11×11×11	1331

Every stage uses a head dimension of **64**, with $C/64$ heads. Window sizes are measured in **feature grid points at the corresponding layer**. NA itself preserves $[C, T, H, W]$; separate modules perform upsampling.

A standard NA Block applies:

```
RMSNorm -> 3D NA Attention -> add residual
RMSNorm -> SwiGLU MLP      -> add residual
```

Q and K use per-head RMSNorm and 3D RoPE; the MLP hidden dimension is **4C**. The Diffusion NA Blocks in Stage 5 additionally inject context and modulate features using the denoising timestep.

All these Attention layers are **non-causal**, allowing attention to subsequent temporal positions. At boundaries, the window shifts into the valid region to retain its size. For example, with a temporal window of 3, the first position attends to 0, 1, 2.

4.5 Downsampling, Upsampling, and Temporal Boundaries

Operation	Implementation	Shape change
Spatial Patchify	Rearrange each frame's 4×4 pixel blocks into the channel dimension	Divide H, W by 4; multiply C by 16; T unchanged
Spatial downsampling	Causal convolution $\rightarrow$ rearrange spatial positions into channels; add a shortcut obtained by rearrangement and grouped averaging	Halve H, W ; T unchanged
Temporal downsampling	Repeat the first frame once $\rightarrow$ causal convolution and temporal rearrangement; add a shortcut	$T \rightarrow (T + 1)/2$; H, W unchanged
Spatiotemporal downsampling	Repeat the first frame once $\rightarrow$ causal convolution and joint $2 \times 2 \times 2$ rearrangement; add a shortcut	$T \rightarrow (T + 1)/2$; halve H, W
CNN upsampling	Convolution produces the required channels $\rightarrow$ rearrange channels into the specified spatial/temporal dimensions	Double the specified axes; remove the first frame after temporal expansion
NA Decoder upsampling	Linear produces the required channels $\rightarrow$ rearrange channels into the specified spatial/temporal dimensions	Double the specified axes; remove the first frame after temporal expansion
Spatial Unpatchify	Rearrange channels back into each frame's 4×4 pixel blocks	Multiply H, W by 4; divide C by 16; T unchanged

The main downsampling branch first adjusts channels through convolution, then folds local spatiotemporal positions into the channel dimension. The shortcut applies the same rearrangement to the input and uses grouped averaging to match the output channel count. Spatiotemporal resampling uses joint rearrangement.

Residual Blocks, Normalization, and Chunking

- **CNN residual blocks:** the main branch primarily applies two sets of "PixelNorm $\rightarrow$ SiLU $\rightarrow 3 \times 3 \times 3$ convolution," then adds the shortcut, preserving T, H, W .
- **PixelNorm:** RMS normalization along the channel dimension at each spatiotemporal position.
- **Temporal boundaries:** the Encoder's causal convolutions use first-frame replication for left temporal padding. Rearrangement modules that include temporal downsampling also repeat the first frame once more. The Decoder removes leading frames after temporal expansion, giving the overall length $T_{\text{out}} = 8(T_z - 1) + 1$.
- **Chunked execution:** LTX supports tiling with overlap regions. Here, $81 = 1 + 10 \times 8$ expresses the temporal compression relationship. Actual decoding must retain context across time; the 11 latent temporal positions cannot simply be decoded independently and concatenated.

5. FLUX 3 Video VAE

Next is the video VAE in FLUX 3 Action. It uses a **causal NA Transformer Encoder + non-causal NA Transformer Decoder**, with a temporal compression factor of **4**, spatial compression factors of **32** along both height and width, and **96** latent channels.

Again, consider an RGB video with **81 frames at 480×832**. Shapes are written as $[C, T, H, W]$, omitting $B=1$:

Full video: $[3, 81, 480, 832] \rightarrow$ **latent:** $[96, 21, 15, 26] \rightarrow$ **reconstructed video:** $[3, 81, 480, 832]$.

$$T_z = 1 + \frac{81 - 1}{4} = 21, \quad H_z = \frac{480}{32} = 15, \quad W_z = \frac{832}{32} = 26.$$

5.1 Blocks and Attention

The FLUX 3 VAE also uses Neighborhood Attention, with **5×5×5** windows throughout, together with QK Norm and 3D RoPE. **The Encoder uses temporally causal Attention, while the Decoder uses non-causal Attention.**

Feature channels	Attention heads	Head dimension	Window: time × height × width
256	4	64	5×5×5
512	8	64	5×5×5
1024	16	64	5×5×5
2048	32	64	5×5×5

Within a single NA layer, the Encoder attends only to **the current temporal position and up to 4 preceding positions**. Away from boundaries, the Decoder attends to **the preceding 2 positions, the current position, and the following 2 positions**; at boundaries, the window shifts toward the sequence interior. Single-frame inputs use **5×5 2D NA**.

5.2 Actual Chunking and Context

Encoding chunks: execution uses **45-frame chunks, advancing by 44 frames each time**. Neighboring chunks overlap by 1 frame, and each chunk enters the Encoder independently. For the classic 81-frame example, the final frame is first repeated to **pad 81 frames to 89**:

Chunk 1: frames 1-45

Chunk 2: frames 45-89

Each chunk produces **12 latent temporal positions**. Keep the entire output of the first chunk, discard the first latent temporal position from subsequent chunks, then concatenate and crop according to the original video length:

$$12 + (12 - 1) = 23 \rightarrow \text{keep the first 21 temporal positions.}$$

5.3 Encoder: Tracing a 45-Frame Video Chunk

The entry divides pixels into **1×4×4** patches and projects them to 256 channels, implemented as a Conv3d with both kernel and stride set to 1×4×4. Three spatial merging steps follow, then two temporal merging steps at low spatial resolution.

Step	Module and operation	Output shape
Input	Current video chunk	$[3, 45, 480, 832]$
Entry	$1 \times 4 \times 4$ Conv3d, stride (1, 4, 4), $3 \rightarrow 256$	$[256, 45, 120, 208]$
Stage 1	NA Block ×1, 256 channels $\rightarrow$ spatial merging, 256→512	$[512, 45, 60, 104]$
Stage 2	NA Block ×4, 512 channels $\rightarrow$ spatial merging, 512→1024	$[1024, 45, 30, 52]$
Stage 3 backbone	NA Block ×8, 1024 channels $\rightarrow$ spatial merging, 1024→2048	$[2048, 45, 15, 26]$

Step	Module and operation	Output shape
First temporal compression	Additional NA Block $\times 1 \rightarrow$ temporal merging, 45$\rightarrow$23	[2048, 23, 15, 26]
Stage 4 backbone	NA Block $\times 8$, 2048 channels	[2048, 23, 15, 26]
Second temporal compression	Additional NA Block $\times 1 \rightarrow$ temporal merging, 23$\rightarrow$12	[2048, 12, 15, 26]
Exit	Position-wise Linear, 2048$\rightarrow$192	[192, 12, 15, 26]

Spatial compression consists of the $4\times$ entry compression and three $2\times$ spatial merging steps:

$$4 \times 2 \times 2 \times 2 = 32.$$

The temporal lengths are:

$$45 \rightarrow 23 \rightarrow 12.$$

The output is split into two groups along the channel dimension, each with shape [96, 12, 15, 26]. Only the first group is taken as μ . The mean μ is then normalized using the per-channel means and standard deviations stored in the model.

5.4 Decoder: Reconstructing the Video Stage by Stage

First undo the latent's per-channel normalization, then reconstruct the RGB video through Linear projections, NA Blocks, and spatiotemporal expansion. Shapes are written as [C, T, H, W], omitting the batch dimension.

Step	Module and operation	Output shape
Input	Latent after undoing normalization	[96, 21, 15, 26]
Entry	Position-wise Linear, 96$\rightarrow$2048	[2048, 21, 15, 26]
Stage 1 temporal expansion	NA Block $\times 8 \rightarrow$ temporal expansion, 21$\rightarrow$41 $\rightarrow$ NA Block $\times 1$	[2048, 41, 15, 26]
Stage 1 spatial expansion	Spatial expansion, 2048$\rightarrow$1024	[1024, 41, 30, 52]
Stage 2 temporal expansion	NA Block $\times 8 \rightarrow$ temporal expansion, 41$\rightarrow$81 $\rightarrow$ NA Block $\times 1$	[1024, 81, 30, 52]
Stage 2 spatial expansion	Spatial expansion, 1024$\rightarrow$512	[512, 81, 60, 104]
Stage 3	NA Block $\times 4 \rightarrow$ spatial expansion, 512$\rightarrow$256	[256, 81, 120, 208]
Stage 4	NA Block $\times 1$, retaining 256 channels	[256, 81, 120, 208]
Exit	Position-wise Linear, 256$\rightarrow$48	[48, 81, 120, 208]
Unpatchify	Rearrange 48=3$\times$4$\times$4 channels into RGB pixel blocks	[3, 81, 480, 832]

Each temporal expansion follows $T \rightarrow 2T - 1$, giving:

$$21 \rightarrow 41 \rightarrow 81.$$

5.5 Downsampling, Upsampling, and First-Frame Alignment

Operation	Implementation	Shape change
Entry Patch Embedding	$1 \times 4 \times 4$ Conv3d, stride $1 \times 4 \times 4$, $3 \rightarrow 256$	Divide H, W by 4; T unchanged
Spatial merging	Concatenate 2×2 neighboring positions: $C \rightarrow 4C \rightarrow$ LayerNorm $\rightarrow$ Linear, $4C \rightarrow 2C$	Halve H, W ; double C
Temporal merging	For odd lengths, repeat the first position at the beginning $\rightarrow$ concatenate pairs of positions $\rightarrow$ LayerNorm $\rightarrow$ Linear, $2C \rightarrow C \rightarrow$ add a shortcut containing the mean of the two positions	$T \rightarrow \lceil T/2 \rceil$; C, H, W unchanged
Spatial expansion	LayerNorm $\rightarrow$ Linear, $C \rightarrow 4C_{\text{out}} \rightarrow$ rearrange into 2×2 spatial positions, where $C_{\text{out}} = C/2$	Double H, W ; halve C
Temporal expansion	LayerNorm $\rightarrow$ Linear, $C \rightarrow 2C \rightarrow$ add a shortcut consisting of two copies of the input along the channel dimension $\rightarrow$ rearrange into the time dimension $\rightarrow$ remove the first position	$T \rightarrow 2T - 1$; C, H, W unchanged
Spatial Unpatchify	Rearrange the 48 channels at each position into a $3 \times 4 \times 4$ pixel block	Multiply H, W by 4; output 3 channels

Execution order:

- Encoder Stage 3: spatial merging $\rightarrow$ additional NA Block $\rightarrow$ temporal merging. Stage 4 applies an additional NA Block $\rightarrow$ temporal merging after its backbone.
- The first two Decoder stages: temporal expansion $\rightarrow$ additional NA Block $\rightarrow$ spatial expansion.

First-frame alignment: encoding pads odd lengths by repeating the first position, while decoding restores alignment by removing the first position after expansion. Single-frame inputs also pass through these modules, but their length remains 1:

Encoder: 1 $\rightarrow$ repeat to 2 $\rightarrow$ merge to 1

Decoder: 1 $\rightarrow$ expand to 2 $\rightarrow$ remove the first position, leaving 1

After two temporal expansion steps, the output length is:

$$T_{\text{out}} = 2(2T_z - 1) - 1 = 4(T_z - 1) + 1.$$